Architecture

Overview

zq parses JSON into a flat tape, compiles jq filters into bytecode, and executes them via a stack-based VM. For JSONL workloads, a fixed-size worker pool splits the input into newline-aligned chunks and processes them in parallel, reordering results before output.

Zero external dependencies. Zig 0.15.2 only.

Data Flow

```
Single-threaded (interactive / small input):

  stdin/file → io.Source → parser.feed() → Tape
                                             ↓
                                query.execute() → ResultIterator.next() → Value
                                                                            ↓
                                                                output.Writer.write_value() → stdo


Parallel - file mode:

  file → mmap → split into ~64 newline-aligned byte-range chunks
                  ↓
          pool.submit_file() → JobQueue (MPMC)
                                  ↓

              worker 0        worker 1         worker N
              (own Parser,    (own Parser,     (own Parser,
               own Iterator)   own Iterator)    own Iterator)


                                  ↓
                       Sequencer ring buffer (reorder)
                                  ↓
                            collect_bytes() → main → stdout


Parallel - stream mode:

  stdin → io.Source → IO thread (accumulates 256 KB batches)
                        ↓
                    JobQueue → workers → Sequencer → collect_bytes() → stdout
```

Modules

```
error   (no deps)          Zig error set, lazy line/col resolution
types   (no deps)          Tape, Value, Format, Instruction, Op
io      → error            mmap (files) / ring buffer (pipes)
parser  → error, types     Streaming state machine → flat Tape
query   → error, types     Compiler + bytecode VM + ResultIterator
output  → error, types     Buffered serialization, generic over sink type
pool    → all above        Worker pool, Sequencer, InFlightLimiter
c_abi   → error, types,    C ABI bridge: zq_compile/zq_execute/zq_free
          parser, query
main    → all              CLI arg parsing, routing
```

Each module's public API, constraints, and invariants: `src/[module]/INTERFACE.md`

error

`ZqError` error set for Zig-native `try/catch` propagation. `Error/Context` structs for display only — built at the outermost boundary, never on the hot path.

Lazy line/column resolution: byte offsets are tracked during parsing. When an error surfaces, a `LineTable` is built (scan for `\n` offsets) and the offset is resolved via binary search. Cost on the happy path: zero.

types

Shared definitions used across all modules: - `Tape` — flat array of `Tag + Payload` entries (the parsed JSON representation) - `Value` — non-owning view into a Tape: tag + payload + span - `Format` — output format enum: pretty, compact, raw, jsonl, join - `Instruction` / `Op` / `Operand` — bytecode format for the query VM - `StringRef` — offset+length into tape's string buffer

io

Unified byte stream. Backend chosen at `init` based on file type: - **mmap** for regular files — zero-copy, no syscalls after init - **Ring buffer** for pipes/stdin/sockets — 64 KB, grows at most once

`peek()/consume()` are pointer arithmetic only. `refill()` is the sole syscall site.

Cross-platform: `std.fs.File` throughout (not POSIX fd_t). Windows uses `CreateFileMappingW/MapViewOfFile` via comptime branching.

parser

Streaming state machine. `feed()` called repeatedly as chunks arrive, maintaining state across calls. Returns `FeedResult.done` (complete value) or

`.need_more` (valid prefix, call again).

Produces a `Tape`: flat array of entries, no pointers, no hash maps. Container open tags store their entry count — O(1) skip.

SIMD scan (AVX2 on x86-64, NEON on AArch64) classifies structural characters 64 bytes at a time. Hidden behind platform-independent interface.

Auto-close: truncated containers at EOF are synthetically closed (`{"a":1` → `{"a":1}`). Unterminated strings always error — can't know the intended endpoint.

Depth limit: 512. Not thread-safe — each worker owns its own instance.

query

Three phases:

1. **Lexer** — tokenizes filter string. Handles string literals, operators, keywords, field names, numbers.
2. **Compiler** — recursive descent → bytecode. Includes a **fuse pass**: `.a | .b | .c` collapsed into a single `load_path` instruction.
3. **VM** — stack-based execution. Filters are generators (0..N outputs per input). Eval stack, value stack, generator stack, try stack, collect stack, if stack.

`CompiledQuery` is immutable after compile — thread-safe for concurrent `execute()` calls. Each `execute()` produces an independent `ResultIterator`.

`ResultIterator.reset(tape)` rebinds to a new tape with zero allocations. Mirrors `Parser.reset()`.

Implemented operators: field access, pipes, iteration, arithmetic, comparisons, boolean logic, conditionals, variables, user-defined functions, string interpolation, recursive descent, alternative (`//`), try/catch, optional (`?`), slicing, update assignment (`|=`, `+=`, etc.), comma (generators), array/object construction, bracket expressions.

Builtins: length, keys, values, has, in, type, empty, select, map, add, not, error, tostring, tonumber, range, sort, sort_by, group_by, unique, unique_by, reverse, flatten, min, max, min_by, max_by, to_entries, from_entries, with_entries, del, contains, inside, any, all, limit, first, last, indices, index, rindex.

`runtime_tape` handles object/array construction — grows monotonically within a `next()` call, cleared only at `reset()`. Self-copy safety: pre-reserves exact capacity and refreshes view before copy loop.

output

64 KB buffered writes to file descriptors. Formats: pretty (2-space indent), compact, raw (unquoted strings), jsonl, join (raw, no trailing newline).

Serialization is generic via Zig's `anytype` — parameterized on `writeByte`/`writeSlice` methods. Same code powers `Writer` (fd-backed, used by main thread) and `BufferSink` (ArrayList-backed, used by pool workers).

ANSI color output when stdout is a TTY. `-C` forces color, `-M` disables, `NO_COLOR` env var respected. Color state shared via `*const Color` pointer across main thread and workers.

pool

Fixed-size worker pool. Orchestrates io → parser → query → output in parallel.

File mode: mmap the file, scan for newline boundaries to split into ~64 byte-range chunks. Workers steal chunks from an MPMC `JobQueue`.

Stream mode: dedicated IO thread reads from `Source`, accumulates complete lines into 256 KB batches (`STREAM_BATCH_SIZE`), posts as Jobs. Flush on pipe stall (peek returns 0 bytes, not EOF) preserves interactive latency.

Execution paths: - Structured (`format=null`): workers copy values into arena-backed `OwnedValue`. Caller uses `collect()`. - Serialized (`format!=null`): workers serialize directly into arena byte buffers while tape is live, bypassing `OwnedValue`. Caller uses `collect_bytes()`.

Sequencer: fixed-size ring buffer. `post()` is a direct array write at `chunk_id % capacity`. `next_in_order()` is a direct array read. Zero dynamic allocations in hot path.

InFlightLimiter: feeder thread acquires a slot before enqueuing each chunk, blocks when `IN_FLIGHT_FACTOR × n_threads` chunks are live. `collect()` releases slots on arena free. Caps peak RSS to a bounded function of thread count, not file size.

Memory model: arena-per-chunk, freed atomically when chunk is fully consumed. Workers own persistent `Parser` + `ResultIterator` — init once, `reset()` per record. 2 MiB thread stacks (reduced from 16 MiB default).

`n_threads = 0` is valid — calling thread executes all work inline.

c_abi

Four exported C functions: `zq_compile`, `zq_execute`, `zq_get_result`, `zq_free`.

Opaque `QueryHandle` owns everything: `CompiledQuery`, `Parser`, null-terminated result buffer. Error codes: 0=ok, -1=parse, -2=query, -3=OOM.

Not thread-safe per handle. Each concurrent caller needs its own handle.

main

CLI entry point. Parses jq-compatible flags into `Config` struct. Routes to: - Pool serialized path when input is a file and format is known - Pool stream path when input is stdin - Single-threaded path as fallback

Flags: `-r` (raw), `-c` (compact), `-e` (exit status), `-n` (null input), `-j` (join output), `-f` (filter from file), `-C`/`-M` (color), `--tab`, `--indent`.

Design Decisions

Tape over tree

Parse JSON into a flat array of `Tag + Payload` entries instead of a tree of heap-allocated nodes.

Trees require one allocation per node, pointer chasing on every traversal, and O(n) skip to walk past a container. A flat tape gives O(1) skip (container open tag stores entry count), sequential memory access (cache-friendly), and zero per-node allocations.

Tradeoff: mutation requires rebuilding. Handled by `runtime_tape` in the query VM — object/array construction writes new entries into a separate growable tape.

Chunk-level parallelism, not record-level

Split files into ~64 byte-range chunks, not one job per JSON record.

Record-level parallelism on a 15M-line file means 15M sequencer operations, 15M arena alloc/free cycles, and 15M job queue interactions. Chunk-level batching reduces all of this to ~64 — same order of magnitude regardless of record count. Each chunk contains thousands of records processed sequentially within a single worker, amortizing orchestration overhead.

Initial implementation enqueued one job per line (2.16M jobs, 38s). Rewrite to chunk-level: 2.24s. Further optimization with InFlightLimiter: 1.41s.

Arena-per-chunk with atomic deallocation

Each chunk gets its own arena allocator, freed atomically when `collect()` / `collect_bytes()` advances past it.

Per-record alloc/free creates millions of GPA round-trips. Arena allocation is O(1) bump-pointer with no per-object bookkeeping. The arena outlives all records in its chunk, so no dangling references within a chunk's lifetime.

Serialized execution path

When the output format is known at submit time, workers serialize values directly into byte buffers while the tape is still live, bypassing `OwnedValue` entirely.

For scalar queries (`.id`, `select()`), `OwnedValue` copies ~150 bytes/record (tape entries + string data). Serialized output is ~3 bytes/record (`"42\n"`). On 15M records: ~700 MB vs ~50 MB of arena memory.

Result: RSS for `.id` query dropped from ~2.6x to ~1.1x input size. Contiguous byte buffer + compact `RecordMeta` array (8 B/record vs ~32 B per `RecordOutcome`) further reduced allocation count.

Implementation detail: `meta_list` is pre-allocated before `chunk_buf` so the chunk buffer (always the arena's last allocation) can resize in-place when output exceeds input size (e.g. pretty format). Uses `.items` directly instead of `toOwnedSlice` — arena owns the memory.

InFlightLimiter backpressure

Feeder thread acquires a slot before enqueuing each chunk, blocks when `IN_FLIGHT_FACTOR × n_threads` chunks are live.

Without backpressure, all chunks are allocated simultaneously. For a 648 MB file: 64 arenas alive at once, RSS ~3 GB. With backpressure (factor=2, 16 threads): at most 32 chunks live, RSS bounded regardless of file size.

Result: 2998 MB → 1764 MB (-41%). Speed also improved from 38s → 1.41s — memory pressure was causing thrashing.

Ring buffer sequencer

Fixed-size ring buffer replaces HashMap-based reorder buffer.

HashMap has allocation overhead per insert and hash collision overhead. Ring buffer is direct array index (`chunk_id % capacity`), O(1) guaranteed, zero dynamic allocations in hot path. Capacity = `max(IN_FLIGHT_FACTOR × n_threads, QUEUE_CAP + n_threads)`. The ring-slot invariant (no two live chunks share an index) is naturally enforced by InFlightLimiter (file mode) and QUEUE_CAP (stream mode).

Stream batching

IO thread accumulates complete lines into 256 KB batches before creating a Job, rather than one Job per line.

Per-line jobs caused 2.16M sequencer operations for a 648 MB file. Batching reduces this to ~2,540 — same order of magnitude as file mode. Flush on pipe stall preserves interactive latency for small/slow inputs.

Result: streaming stdin 215s → 1.8s (120x faster), 7 MB RSS.

Auto-close for truncated JSON

When `is_eof=true` and the parser's structural stack is non-empty, synthetically close all open containers and return `.done` instead of error.

LLM streaming produces truncated JSON mid-response (`{"tool": "search", "args": {`). jq crashes. zq auto-closes to `{"tool": "search", "args": {}}` and keeps going.

Unterminated strings are the exception — the parser cannot know where the string was supposed to end. Always returns `error.UnterminatedString`.

Lazy error resolution

Track byte offsets during SIMD tokenization. Compute line/column numbers only on the error path.

Counting `\n` during tokenization costs ~1-2% CPU on every record. Errors are rare — one per millions of records in production. A `LineTable` (array of `\n` offsets, binary search) is built only when an error needs to be surfaced. Cost on the happy path: zero.

Fuse pass

Post-compilation pass collapses `.a | .b | .c` chains into single `load_path` instructions.

Without fusion: 3 opcode dispatches with value stack push/pop between each. Fused: single opcode walks the tape directly. This is the most common filter pattern in real-world jq usage. The fuse pass also updates the instruction index map so that jump targets remain valid after collapsing.

i64 integers, not f64

Store integers as i64 in the tape. Exact to 2^63 - 1.

jq uses f64 for all numbers, silently corrupting integers above 2^53. API responses with snowflake IDs, database primary keys, or blockchain values get mangled. i64 preserves exact values. Arithmetic overflow returns an error instead of silently wrapping.

Deliberate deviation from jq.

IEEE 754 division semantics

`0/0 = nan`, `n/0 = infinite` (n>0), `n/0 = -infinite` (n<0).

jq's division-by-zero behavior is inconsistent across versions and platforms. IEEE 754 is well-defined and portable.

Deliberate deviation from jq.

Non-owning views everywhere

All cross-module data references — `SliceView`, `snippet`, `Tape`, `Value` — are non-owning views into caller-managed memory.

Eliminates copy overhead and ownership ambiguity. Every module follows the same contract: view is valid until backing memory is freed/recycled. Documented consistently across all INTERFACE.md files.

Per-worker persistent state

Each worker thread inits a `Parser` and `ResultIterator` once, then calls `reset()` per record.

`Parser.init()` involves 3+ allocations (tape buffer, string buffer, structural stack). `ResultIterator` via `execute()` involves 6 allocations (eval stack, value stack, etc.). On 15M records, resetting instead of init/deinit avoids ~135M alloc/free cycles.

Cross-platform via std.fs.File

Replaced `std.posix.fd_t` with `std.fs.File` throughout io, output, pool, and main modules.

Enables Windows compilation without POSIX shims. Platform-specific APIs (mmap vs CreateFileMappingW, posix.getenv vs process.getenvW) selected via comptime branching — no runtime dispatch.

Generic serialization via anytype

Output serialization functions take `anytype` requiring `writeByte`/`writeSlice` methods.

Same serialization code powers both `Writer` (fd-backed, 64 KB buffered, used by main thread) and `BufferSink` (ArrayList-backed, used by pool workers for serialized path). No code duplication, no vtable overhead — monomorphized at comptime.

2 MiB thread stacks

Reduced worker thread stack size from 16 MiB default to 2 MiB.

Workers use heap-allocated buffers for parse/query work. 2 MiB gives 4x margin over measured worst-case stack recursion. On 16 threads: 224 MB saved.

Single source of truth for version

Version defined in `build.zig` with `-Dversion` override for release CI. Install scripts, Homebrew formula, AUR PKGBUILD, and release workflow all derive from this.

Deliberate Deviations from jq

Behavior	jq	zq	Rationale
Large integers	f64, loses precision above 2^53	i64, exact to 2^63	Data integrity
Division by zero	Inconsistent	IEEE 754 (nan/infinite)	Consistency
Truncated JSON	Crashes	Auto-close containers	LLM streaming
Duplicate keys	Silent last-wins	Last-wins + optional `--warn-duplicate-keys`	Data integrity

Performance Snapshot

648 MB / 15M-record JSONL, `.id` query:

```
File mode:     0.87s,  715 MB RSS  (25x faster than jq, 1.10x input size)
Stream mode:   1.4s,     7 MB RSS  (16x faster than jq)
Startup:       ~2 ms               (2x faster than jq)
Binary:        2.7 MB              (static, stripped, zero deps)
```

Memory optimization progression:

```
Initial (all chunks live):        2998 MB
+ InFlightLimiter:                1764 MB  (-41%)
+ Ring buffer sequencer:          1702 MB  (-3.5%)
+ Serialized path:                1124 MB  (-34%)
+ 2 MiB stacks + contiguous:      792 MB  (-29%)
+ Arena interleaving fix:          715 MB  (-10%)
                          Total:          -76%
```